



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 2

по курсу «Разработка параллельных и распределенных программ»

Студент: Ивенкова О. В.

Группа: ИУ9-52Б

Преподаватель: Царёв А. С.

Москва, 2025

Содержание

1	Постановка задачи	3
2	Практическая реализация	5
3	Сравнение вычислений	10
4	Характеристики устройства	11

1 Постановка задачи

Лабораторная работа №2: «Параллельная реализация решения системы линейных алгебраических уравнений с помощью MPI».

РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ

Пусть есть система из N линейных алгебраических уравнений в виде $Ax = b$, где A – матрица коэффициентов уравнений размером $N \times N$, b – вектор правых частей размером N , x – искомый вектор решений размером N . Решение системы уравнений итерационным методом состоит в выполнении следующих шагов.

1. Задается x_0 – произвольное начальное приближение решения (вектор с произвольными начальными значениями).

2. Приближение многократно улучшается с использованием формулы вида $x_{n+1} = f(x_n)$, где функция f определяется используемым методом.

3. Процесс продолжается, пока не выполнится условие $g(x_n) < \varepsilon$, где функция g определяется используемым методом, а величина ε задает требуемую точность.

Персональный вариант: Метод сопряженных градиентов.

Метод сопряженных градиентов

В методе сопряженных градиентов преобразование решения на каждом шаге задается следующими формулами:

$$\begin{aligned} r^0 &= b - Ax^0, \\ z^0 &= r^0, \\ \alpha^{n+1} &= \frac{(r^n, r^n)}{(Az^n, z^n)}, \\ x^{n+1} &= x^n + \alpha^{n+1} z^n, \\ r^{n+1} &= r^n - \alpha^{n+1} Az^n, \\ \beta^{n+1} &= \frac{(r^{n+1}, r^{n+1})}{(r^n, r^n)}, \\ z^{n+1} &= r^{n+1} + \beta^{n+1} z^n, \end{aligned} \tag{1}$$

где $(u, v) = \sum_{i=0}^{N-1} u_i v_i$. Метод сопряженных градиентов на каждой итерации содержит только одно умножение матрицы на вектор и, кроме того, сходится

быстрее, чем предыдущие два метода. Однако метод сопряженных градиентов работает только для симметричных матриц коэффициентов. Критерий завершения счета в методе сопряженных градиентов следующий:

$$\frac{\|r^n\|_2}{\|b\|_2} < \varepsilon, \quad \text{где} \quad \|u\|_2 = \sqrt{\sum_{i=0}^{N-1} u_i^2} \quad (2)$$

Подбирать значения параметра ε лучше всего исходя из задачи, начиная со значения 0.00001. Если, начиная с определенной итерации значение этого параметра перестает меняться, значит, можно выбрать в качестве критерия завершения счёта именно это установившееся значение.

ИСХОДНЫЕ ДАННЫЕ

Исходные данные для тестирования реализаций представленных методов и выполнения лабораторной работы можно взять следующие.

1. Модельная задача с заданным решением

Элементы главной диагонали матрицы A равны 2.0, остальные равны 1.0. Все элементы вектора b равны $N + 1$. В этом случае решением системы будет вектор, элементы которого равны 1.0. Начальные значения элементов вектора x можно взять равными 0.

2. Модельная задача с произвольным решением

Элементы главной диагонали A равны 2.0, остальные равны 1.0. Формируется вектор u , элементы которого заполняются произвольными значениями, например $u_i = \sin \frac{2\pi i}{N}$. Элементы вектора b получаются путем умножения матрицы A на вектор u . В этом случае решением системы будет вектор, равный вектору u . Начальные значения элементов вектора x можно взять равными 0.

ЗАДАНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ

1. Написать программу, которая реализует итерационный алгоритм решения системы линейных алгебраических уравнений вида $Ax = b$ в соответствии с выбранным вариантом. Здесь A – матрица размером $N \times N$, x и b – векторы длины N . Тип элементов – double.

2. Программу распараллелить с помощью MPI с разрезанием матрицы A по строкам на близкие по размеру, возможно не одинаковые, части. Соседние

строки матрицы должны располагаться в одном или в соседних MPI-процессах.

Реализовать два варианта программы:

1: векторы x и b дублируются в каждом MPI-процессе,

2: векторы x и b разрезаются между MPI-процессами аналогично матрице A . (только для сдающих после срока)

Уделить внимание тому, чтобы при запуске программы на различном числе MPI-процессов решалась одна и та же задача (исходные данные заполнялись одинаковым образом).

3. Замерить время работы двух вариантов программы при использовании различного числа процессорных ядер: 1, 2, 4, 8, 16. Построить графики зависимости времени работы программы, ускорения и эффективности распараллеливания от числа используемых ядер. Исходные данные, параметры N и ε подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд. Также параметр N разрешено подобрать таким образом, чтобы он нацело делился на 1, 2, 4, 8 и 16.

4. На основании полученных результатов сделать вывод о целесообразности использования одного или второго варианта программы.

2 Практическая реализация

Листинг 1: Код программы

```
1 #include <mpi.h>
2 #include <vector>
3 #include <cmath>
4 #include <iostream>
5
6 using std::vector;
7
8 int mpi_rank, mpi_size;
9 int n;
10 int local_n;
11 int local_start;
12
13 double dotProduct(const vector<double>& v1, const vector<double>& v2) {
14     double local_sum = 0.0;
15     for (int i = 0; i < v1.size(); ++i) local_sum += v1[i] * v2[i];
16 }
```

```

17     double global_sum = 0.0;
18     MPI_Allreduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM,
19     MPI_COMM_WORLD);
20     return global_sum;
21 }
22 double norm(const vector<double>& v) {
23     return std::sqrt(dotProduct(v, v));
24 }
25
26 vector<double> multiplyA(const vector<vector<double>>& A_local, const
27     vector<double>& x,
28     const vector<int>& recvcnts, const vector<int>& displs) {
29     vector<double> local_res(local_n, 0.0);
30     for (int i = 0; i < local_n; ++i) {
31         double sum = 0.0;
32         for (int j = 0; j < n; ++j) sum += A_local[i][j] * x[j];
33         local_res[i] = sum;
34     }
35
36     vector<double> global_res(n);
37     MPI_Allgatherv(local_res.data(), local_n, MPI_DOUBLE,
38         global_res.data(), recvcnts.data(), displs.data(), MPI_DOUBLE,
39         MPI_COMM_WORLD);
40     return global_res;
41 }
42
43 vector<double> solveCG(const vector<vector<double>>& A_local, vector<
44     double> x, const vector<double>& b,
45     double eps, const vector<int>& recvcnts, const vector<int>& displs
46 ) {
47
48     vector<double> r(local_n, 0.0);
49     vector<double> z(local_n, 0.0);
50
51     vector<double> Ax = multiplyA(A_local, x, recvcnts, displs);
52     for (int i = 0; i < local_n; ++i) {
53         r[i] = b[local_start + i] - Ax[local_start + i];
54         z[i] = r[i];
55     }
56
57     double rr_old = dotProduct(r, r);
58     double norm_b = std::sqrt(dotProduct(vector<double>(b.begin() + 0, b
59     .begin() + n),
60     vector<double>(b.begin() + 0, b.begin() + n)));

```

```

58     if (norm_b == 0.0) norm_b = 1.0;
59
60     int max_iter = n;
61     for (int iter = 0; iter < max_iter; ++iter) {
62
63         vector<double> z_global(n, 0.0);
64         MPI_Allgatherv(z.data(), local_n, MPI_DOUBLE, z_global.data(),
65             recvcnts.data(), displs.data(), MPI_DOUBLE, MPI_COMM_WORLD
66         );
67
68         vector<double> Az = multiplyA(A_local, z_global, recvcnts,
69             displs);
70
71         vector<double> Az_local(local_n, 0.0);
72         for (int i = 0; i < local_n; ++i) Az_local[i] = Az[local_start +
73             i];
74
75         double Az_z = dotProduct(Az_local, z);
76         if (std::isnan(Az_z) || std::isinf(Az_z)) {
77             if (mpi_rank == 0) std::cerr << "ERROR: Az_z is NaN/Inf at
78             iter " << iter << "\n";
79             break;
80         }
81         if (Az_z == 0.0) {
82             if (mpi_rank == 0) std::cerr << "INFO: Az_z == 0 at iter "
83             << iter << " (breaking)\n";
84             break;
85         }
86
87         double alpha = rr_old / Az_z;
88
89         for (int i = 0; i < local_n; ++i) {
90             x[local_start + i] += alpha * z[i];
91             r[i] -= alpha * Az_local[i];
92         }
93
94         double rr_new = dotProduct(r, r);
95         double rel_res = std::sqrt(rr_new) / norm_b;
96
97         if (rel_res < eps) {
98             if (mpi_rank == 0) std::cout << "Converged at iter " << iter
99             << " rel_res=" << rel_res << std::endl;
100             break;
101         }
102
103         double beta = rr_new / rr_old;
104         for (int i = 0; i < local_n; ++i) z[i] = r[i] + beta * z[i];

```

```

98
99     rr_old = rr_new;
100 }
101
102     return x;
103 }
104
105 vector<vector<double>> generateLocalMatrix(int local_n, int n, int
    start_row) {
106     vector<vector<double>> A(local_n, vector<double>(n, 1.0));
107     for (int i = 0; i < local_n; ++i)
108         A[i][start_row + i] = 2.0;
109     return A;
110 }
111
112 int main(int argc, char** argv) {
113     MPI_Init(&argc, &argv);
114     MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
115     MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
116
117     n = 36864;
118     local_n = n / mpi_size;
119
120     int base = n / mpi_size;
121     int rem = n % mpi_size;
122     local_n = base + (mpi_rank < rem ? 1 : 0);
123     local_start = base * mpi_rank + (mpi_rank < rem ? mpi_rank : rem);
124
125     vector<int> recvcnts(mpi_size);
126     vector<int> displs(mpi_size);
127     for (int i = 0; i < mpi_size; ++i) {
128         recvcnts[i] = base + (i < rem ? 1 : 0);
129         displs[i] = base * i + (i < rem ? i : rem);
130     }
131
132     MPI_Barrier(MPI_COMM_WORLD);
133     double t_start = MPI_Wtime();
134
135     vector<vector<double>> A_local = generateLocalMatrix(local_n, n,
        local_start);
136     vector<double> b(n, n + 1.0);
137     vector<double> x(n, 1000.0);
138
139     double eps = 1e-22;
140     x = solveCG(A_local, x, b, eps, recvcnts, displs);
141

```



```

142 MPI_Barrier(MPI_COMM_WORLD);
143 double t_end = MPI_Wtime();
144
145 if (mpi_rank == 0) {
146     std::cout << "Total time = " << t_end - t_start << " sec" << std
::endl;
147
148     double err = 0.0;
149     for (int i = 0; i < n; ++i) {
150         double d = x[i] - 1.0;
151         err += d * d;
152     }
153 }
154
155 MPI_Finalize();
156 return 0;
157 }

```

3 Сравнение вычислений

Листинг 2: Вывод после запуска кода на 1, 2, 4, 8, 16 процессорных ядрах

```
1 PS \mpi_test\x64\Debug> mpiexec -n 1 .\mpi_test.exe
2 Converged at iter 2 rel_res=7.32151e-34
3 Total time = 58.6747 sec
4 PS \mpi_test\x64\Debug> mpiexec -n 2 .\mpi_test.exe
5 Converged at iter 1 rel_res=4.4679e-26
6 Total time = 23.5266 sec
7 PS \mpi_test\x64\Debug> mpiexec -n 4 .\mpi_test.exe
8 Converged at iter 1 rel_res=8.846e-25
9 Total time = 15.7773 sec
10 PS \mpi_test\x64\Debug> mpiexec -n 8 .\mpi_test.exe
11 Converged at iter 1 rel_res=2.03087e-26
12 Total time = 14.2433 sec
13 PS \mpi_test\x64\Debug> mpiexec -n 16 .\mpi_test.exe
14 Converged at iter 1 rel_res=6.46217e-25
15 Total time = 14.3276 sec
```

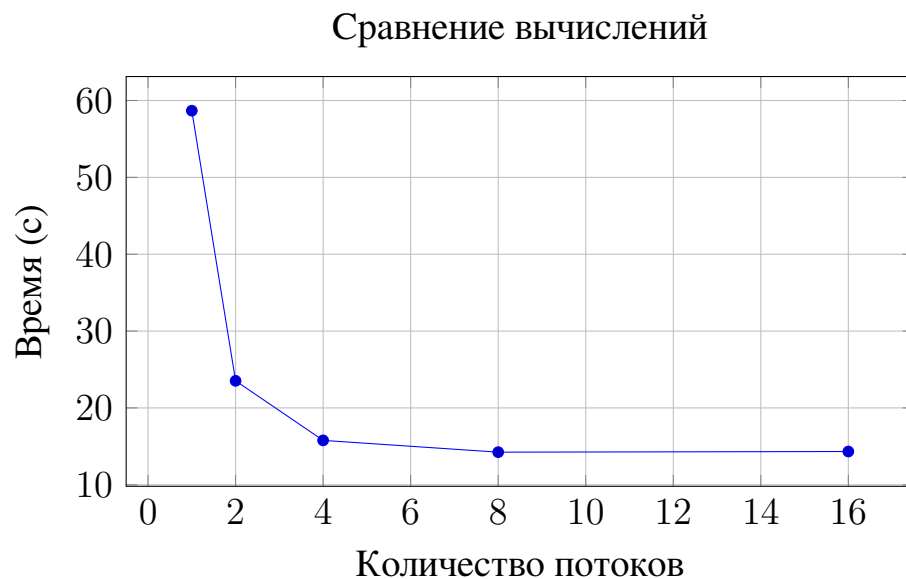


Рис. 1 — Сравнение времени выполнения с разным количеством процессорных ядер

4 Характеристики устройства

Имя устройства Acer Extensa 215-54

Оперативная память 16,0 ГБ

Процессор устройства Intel Core i5-1135G7 (2.40 ГГц, 4 ядра, 15 Вт)